

Agent Study 技术路线

1. 学习目标

从第一性原理搭建 Agent，弄清概率模型和确定性 Python 程序之间的每一道边界。最终系统应当能够规划任务、调用白名单工具、维护显式状态和记忆、安全终止、发现能力回归，并通过 telemetry 解释一次运行。

真正的完成标准不是“仓库能运行”，而是：

- 能脱离代码画出完整数据流；
- 能指出每个失败可能发生的位置；
- 能区分模型生成、Agent 调度和 Python 执行；
- 不依赖 Agent 框架重建核心部件。

2. 核心技术路线

GGUF 模型权重

-> llama.cpp 推理引擎

-> llama-cpp-python Python 接口

-> LocalLLM 的 text in / text out 封装

-> Agent 的 prompt 和输出契约

-> JSON 解析、校验和重试

-> 决策路由与工具请求

-> Python 执行真实工具

-> loop、state、observation 和终止条件

-> memory 与 planning

-> atomic action 与依赖图

-> golden evals

-> telemetry

-> 可验证的本地 Agent

3. 职责边界

部件	主要职责
GGUF 模型	保存训练权重，根据上下文生成概率 token
llama-cpp-python	把 llama.cpp 推理能力暴露给 Python
shared/llm.py	加载模型，统一文本生成接口
agent/agent.py	构造 prompt，对外提供 Agent 能力并调度其他模块
shared/utils.py	把模型文本解析为 Python 可以处理的数据
agent/tools.py	定义 Python 允许执行的白名单动作
agent/state.py	保存步骤、完成状态和终止信息
agent/memory.py	在模型权重之外保存信息

部件	主要职责
agent/planner.py	把计划、原子动作和依赖表示为数据
agent/evals.py	比较实际行为和预期行为
agent/telemetry.py	记录一次运行实际发生了什么

模型负责提出候选文本、决策和结构化参数。Python 负责解析、校验、授权、真实执行、存储、重试和终止。

4. 学习阶段

阶段	课程	技术演进	阶段验收
A. 模型接口	01-03	文本生成 -> 角色 prompt -> JSON 契约	能解释 token、context、解析、校验和重试
B. 最小 Agent	04-06	路由 -> 工具 -> loop 和 state	能区分“请求动作”和“执行动作”，能解释所有终止路径
C. 任务系统	07-10	memory -> plan -> atomic action -> 依赖图	能把任务进度表示为可检查的 Python 数据
D. 可靠性	11-12	golden evals -> telemetry	能发现回归，并还原一次失败运行
E. 综合实践	Capstone	连接以上所有部件	完成有 Eval 和 trace 证据的多步任务

5. 十二课能力地图

Lesson	新增能力	主要源码链路
01	本地 LLM 文本生成	Agent.simple_generate() -> LocalLLM.generate()
02	角色与行为约束	Agent.generate_with_role()
03	结构化输出	Agent.generate_structured() -> extract_json_from_text()
04	有限决策路由	Agent.decide()
05	工具请求与执行	Agent.request_tool() -> execute_tool()
06	循环与显式状态	Agent.run_loop() -> AgentState
07	模型外短期记忆	Agent.run_with_memory() -> Memory
08	顺序计划数据	Agent.create_plan() -> planner.create_plan()
09	带参数的原子动作	Agent.create_atomic_action()
10	依赖图执行	Agent.create_aot_plan() -> execute_graph()
11	回归评测	AgentEval -> evals/golden_datasets.py
12	运行时可观测性	Telemetry -> JSONL trace

6. 后续里程碑

Milestone 1: 完成 Lesson 11

1. 在 `lesson_11_practice.py` 中重新手敲 `test.py` 里的最小 Decision Eval。
2. 至少运行一个通过案例和一个失败案例。
3. 增加 structured output、tool call 和 memory cycle 测试。
4. 不看代码解释 `input`、`expected`、`actual`、`passed` 和 `error`。
5. 分开记录模型失败与评测器自身缺陷。

Milestone 2: 完成 Lesson 12

1. 跟踪一次结构化输出调用和一次工具调用。
2. 让相关 span 共享同一个 trace ID。
3. 记录耗时、状态、重试和失败详情。
4. 读取 `agent_telemetry.jsonl`，还原调用顺序。
5. 保证运行日志不提交到 Git。

Milestone 3: 综合项目

构建一个本地学习助手 Agent：

用户学习目标

- > 生成并校验计划
- > 转换成原子动作或依赖图
- > 调用白名单文本、计算器和笔记工具
- > 工具结果成为下一轮 observation
- > 更新 state 和 memory
- > 因 done、error 或 max_steps 终止
- > Eval 验证关键行为
- > Telemetry 记录完整 trace

最低验收标准：

- 每个模型输出都有解析和校验边界；
- 每个真实动作都经过 Python 白名单；
- loop 具有 `done`、错误和 `max_steps` 终止路径；
- 明确区分进程内 memory 与持久化 memory；
- 至少三个 golden case 覆盖路由、工具和终止；
- 可以通过一个 trace 还原 LLM、工具和状态变化。

Milestone 4: 走向实际工程

综合项目跑通后，再评估持久化、异步执行、并发、沙箱、更严格的 schema 和 Agent 框架。它们是工程扩展，不是理解核心循环的前置条件。